Department of Information Technology

# DATABASE MANAGEMENT SYSTEM

## Mini Project

*Hostel Allotment and Mess Management System*

Yashvika Dhawal
*Enrollment number: 14214803124*
*Email:* yashvikadhawal110@gmail.com

## Abstract

The Hostel Allotment and Mess Management System is a database-driven application designed to automate and streamline the assignment of hostel rooms and the management of mess facilities in an academic institution. The system enforces rule-based allocation constraints including gender-specific hostel assignment, room-capacity limits, department-based eligibility, and active-allotment prerequisites for mess enrollment.

Built using MySQL as the relational database backend and Java with JDBC for application-layer connectivity, the system manages student registrations, room allotments, mess enrollments, weekly meal menus, and fee payment records. Data integrity is upheld through primary keys, foreign keys, BEFORE INSERT triggers, and stored-procedure logic. The project demonstrates the practical application of core DBMS concepts normalisation, ER modelling, constraint design, multi-table JOIN queries, and transactional control in a realistic institutional scenario.


## 1. Introduction

Educational institutions with residential facilities face significant operational challenges in managing hostel allotments and dining services for large student populations. Manual processes are error-prone, time-consuming, and unable to consistently enforce institutional policies. A well-designed relational database system addresses these challenges by providing a structured, automated, and auditable framework.

This project, Hostel Allotment and Mess Management System, automates room allocation, mess enrollment, fee tracking, and menu management. The technology stack is detailed below.

| Component | Details |
|---|---|
| Database | MySQL 8.x — Storage, constraints, triggers, stored procedures |
| Backend Language | Java (JDK 17) — Application logic and business rule processing |
| Connectivity Layer | JDBC API — Interface between Java application and MySQL server |
| Tools | IntelliJ IDEA / Eclipse, MySQL Workbench 8.0 |

The system centres on rule-based allocation constraints to ensure fair, consistent, and policy-compliant assignment of rooms and mess facilities throughout the academic year.

## 2. Problem Statement

In most academic institutions, hostel allotment and mess management are handled through manual or semi-automated processes, leading to several recurring problems:

- Room double-allocation caused by a lack of real-time occupancy tracking.
- Violation of gender-segregation policies when no automated enforcement mechanism exists.
- Inability to prioritise students on the basis of academic year, disability status, or departmental eligibility.
- Inaccurate mess enrollment records leading to overbooking or under-provision of meals.
- Delayed fee payment notifications, causing administrative backlogs and revenue loss.
- Absence of a centralised, query-able history of allotments for institutional audit purposes.

This project addresses the above issues by implementing a structured relational database with rule-enforcing constraints, BEFORE INSERT triggers, and a Java-JDBC backend that validates business rules prior to database commits.

## 3. Objectives

1. Design a normalised (3NF) relational database schema for hostel and mess management in MySQL.
2. Implement rule-based allocation constraints: gender restriction, capacity limits, and year-of-study eligibility.
3. Develop a Java-JDBC backend performing CRUD operations and enforcing business logic via Prepared Statements.
4. Automate room-status updates using BEFORE INSERT triggers on the Allotment table.
5. Manage mess enrollment with configurable meal plans and weekly menu records.
6. Track and report fee payment status for both hostel and mess dues.
7. Demonstrate practical use of stored procedures, SQL views, and aggregate query design.

# 4. Methodology

## 4.1 Requirement Analysis

Institutional hostel rules and mess management policies were analysed to identify entities, relationships, and business constraints. Key rules include: one active allotment per student at any time; room capacity must not be exceeded; hostel type must match student gender; mess enrollment is restricted to students with an active allotment.

## 4.2 Database Design

An ER diagram was constructed to model eight entities and their associations. The schema was normalised to Third Normal Form (3NF) to eliminate redundancy. Tables were created with primary keys, foreign keys, NOT NULL constraints, ENUM/CHECK constraints, and BEFORE INSERT triggers.

## 4.3 Backend Development — Java + JDBC

Java service classes (StudentService, AllotmentService, MessService, FeeService) were developed to interact with MySQL via JDBC. PreparedStatements were used for all DML operations to prevent SQL injection. Business rules — such as gender validation before allotment — are enforced at the application layer prior to database writes. Transactions are managed using conn.setAutoCommit(false) with explicit commit/rollback.

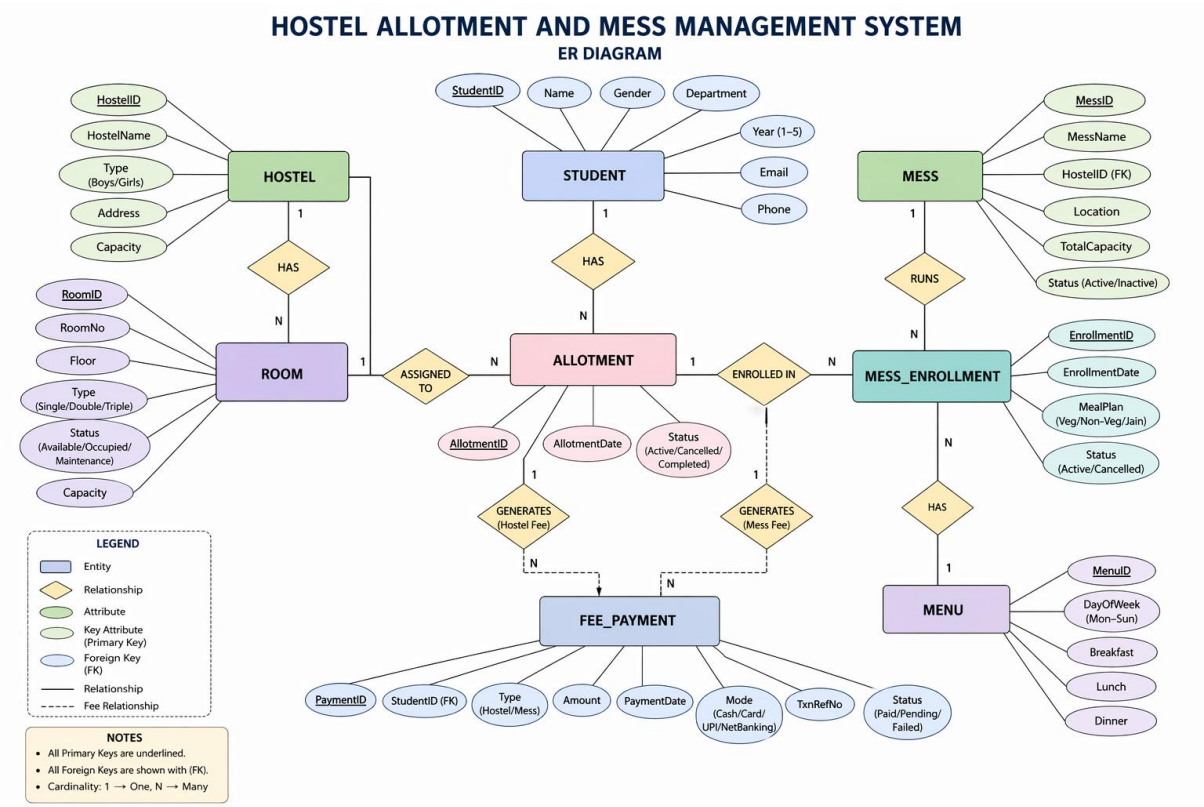
## 4.4 Testing and Validation

SQL queries were tested in MySQL Workbench. JDBC integration was verified using unit test cases. Edge cases including allotment of a full room, enrolling an unallotted student in mess, and duplicate payment entries were tested against expected constraint violations.

# 5. Database Design

## 5.1 Entity-Relationship Diagram

The system comprises eight core entities. Primary keys (PK) uniquely identify each record; foreign keys (FK) establish referential integrity between tables.




## 5.2 Table Summary

| Table | Primary Key | Key Constraints / FK References |
|---|---|---|
| STUDENT | StudentID (Auto Inc.) | Gender ENUM; Year CHECK (1–5) |
| HOSTEL | HostelID | Type ENUM ('Boys','Girls') |
| ROOM | RoomID | HostelID → HOSTEL; Status ENUM |
| ALLOTMENT | AllotmentID | StudentID → STUDENT; RoomID → ROOM; Trigger-checked |
| MESS | MessID | HostelID → HOSTEL |
| MESS_ENROLLMENT | EnrollmentID | StudentID → STUDENT; MessID → MESS; MealPlan ENUM |
| MENU | MenuID | MessID → MESS; DayOfWeek ENUM (Mon–Sun) |

| Table | Primary Key | Key Constraints / FK References |
|---|---|---|
| FEE_PAYMENT | PaymentID | StudentID → STUDENT; Type ENUM ('Hostel','Mess') |

## 5.3 Rule-Based Allocation Constraints

- Gender Rule: Hostel Type must match student Gender before any allotment is permitted.
- Capacity Rule: Active allotment count for a room must be less than its Capacity before insertion.
- Single Allotment Rule: A student may not hold more than one active allotment at any time.
- Active Allotment Prerequisite: Mess enrollment is only permitted for students with Status = 'Active' allotment.
- Mess Capacity Rule: Total active mess enrollments must not exceed the Mess.Capacity value.

# 6. Implementation

## 6.1 DDL — Table and Trigger Definitions

```
-- Student Table
CREATE TABLE Student (
  StudentID   INT PRIMARY KEY AUTO_INCREMENT,
  Name        VARCHAR(100) NOT NULL,
  Gender      ENUM('Male','Female') NOT NULL,
  Department  VARCHAR(60), Year INT CHECK (Year BETWEEN 1 AND 5),
  ContactNo   VARCHAR(15), Email VARCHAR(100)
);

-- Room Capacity Trigger
CREATE TRIGGER trg_check_capacity BEFORE INSERT ON Allotment
FOR EACH ROW BEGIN
  IF (SELECT COUNT(*) FROM Allotment WHERE RoomID = NEW.RoomID
      AND Status = 'Active') >=
     (SELECT Capacity FROM Room WHERE RoomID = NEW.RoomID) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Room capacity exceeded';
  END IF;
END;
```

## 6.2 Java JDBC — Allotment with Gender Rule

```
// Establish Connection
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/hosteldb", "root", "password");

// Step 1: Validate gender rule before allotment
String sql = "SELECT h.Type, s.Gender FROM Student s"
  + " JOIN Room r ON r.RoomID=? JOIN Hostel h ON h.HostelID=r.HostelID"
  + " WHERE s.StudentID=?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setInt(1, roomId); ps.setInt(2, studentId);
ResultSet rs = ps.executeQuery();
if (!hostelType.equalsIgnoreCase(gender + 's'))
  throw new Exception("Gender-hostel mismatch. Allotment denied.");

// Step 2: Insert allotment inside a transaction
conn.setAutoCommit(false);
```

```
String ins = "INSERT INTO Allotment (StudentID,RoomID,DateAllotted,Status)"
           + " VALUES (?,?,CURDATE(),'Active')";
PreparedStatement psi = conn.prepareStatement(ins);
psi.setInt(1, studentId); psi.setInt(2, roomId);
psi.executeUpdate(); conn.commit();
```

## 7. Results

The following SQL queries were executed to validate system functionality. Each query and its output interpretation are described below.

### Query 1 — All Active Allotments with Student and Room Details

```
SELECT s.Name, s.Department, r.RoomNumber, h.HostelName, a.DateAllotted
FROM   Allotment a
JOIN   Student s ON a.StudentID = s.StudentID
JOIN   Room    r ON a.RoomID    = r.RoomID
JOIN   Hostel  h ON r.HostelID  = h.HostelID
WHERE  a.Status = 'Active'
ORDER  BY h.HostelName, r.RoomNumber;
```

Result: Produces a consolidated list of all currently housed students, sorted by hostel and room number. Verifies correct three-table JOIN linkages and confirms that only active allotments are returned. Used by wardens for daily occupancy reporting.

### Query 2 — Rooms with Available Capacity

```
SELECT r.RoomNumber, h.HostelName, r.Capacity,
       (r.Capacity - COUNT(a.AllotmentID)) AS AvailableSlots
FROM   Room r
LEFT   JOIN Allotment a ON r.RoomID = a.RoomID AND a.Status = 'Active'
JOIN   Hostel h ON r.HostelID = h.HostelID
GROUP  BY r.RoomID
HAVING AvailableSlots > 0;
```

Result: Returns rooms with at least one vacant slot. The derived column AvailableSlots is computed dynamically, eliminating stale cached values. This query directly supports capacity-rule enforcement during new allotment requests.

### Query 3 — Students with Pending Fee Payments

```
SELECT s.Name, s.Department, f.Type, f.Amount, f.PaymentDate
FROM   FeePayment f
JOIN   Student s ON f.StudentID = s.StudentID
WHERE  f.Status = 'Pending'
ORDER  BY f.PaymentDate;
```

Result: Lists all students with outstanding hostel or mess fees in chronological order of due date. This output is used by the accounts department to generate payment reminders and validates the FeePayment.Status ENUM constraint.

### Query 4 — Mess Enrollment vs. Capacity Summary

```
SELECT m.MessName, m.Capacity,
       COUNT(e.EnrollmentID)            AS Enrolled,
       (m.Capacity - COUNT(e.EnrollmentID)) AS Remaining
FROM   Mess m
LEFT   JOIN MessEnrollment e ON m.MessID = e.MessID
GROUP  BY m.MessID;
```

Result: Summarises each mess facility's total capacity, current enrollment count, and remaining seats. Demonstrates GROUP BY with derived aggregate columns. Directly supports the mess-capacity allocation rule and assists the mess incharge in managing meal preparation quantities.

## 8. Conclusion

The Hostel Allotment and Mess Management System successfully demonstrates the design and implementation of a rule-based, relational database application for academic institutions. The project achieves all stated objectives by combining a well-normalised MySQL schema with a Java-JDBC backend that enforces allocation policies at multiple architectural layers.

Key technical outcomes include the creation of eight interrelated tables in 3NF, BEFORE INSERT triggers for real-time capacity enforcement, PreparedStatement-based JDBC for secure data access, and multi-table JOIN and aggregation queries that produce actionable administrative reports. The rule-based constraints collectively ensure that no student is assigned to a gender-mismatched hostel, no room exceeds its occupancy limit, no student holds duplicate active allotments, and mess enrollment remains within declared capacity.

This system provides a robust, extensible foundation for a production-grade institutional hostel management solution and reinforces the practical value of relational database design principles in real-world problem-solving.

## 9. Future Scope

- Web-Based Interface: Migrate the console-based Java frontend to a Spring Boot REST API with a React.js web portal accessible to students and administrators via browser.
- Auto-Allotment Engine: Implement a priority-based allotment algorithm that ranks students by academic year, CGPA, and special needs, and assigns rooms automatically at semester start.
- Notification System: Integrate email and SMS alerts via JavaMail API or Twilio for allotment confirmations, fee due reminders, and weekly menu broadcasts.
- Biometric / RFID Mess Attendance: Connect mess enrollment with RFID or fingerprint readers to track actual daily meal consumption and auto-compute monthly fee dues.
- Analytics Dashboard: Generate occupancy, fee-collection, and mess-utilisation reports as interactive charts using JFreeChart or a Tableau integration.
- Cloud Deployment: Host the database on AWS RDS or Azure Database for MySQL, containerise the backend with Docker, and enable institution-wide scalability.
- Mobile Application: Develop Android and iOS companion apps allowing students to check allotment status, view daily mess menus, and pay fees online.

## References


- Ramakrishnan, R. and Gehrke, J. (2003) Database Management Systems. 3rd edn. New York: McGraw-Hill Education.
- Silberschatz, A., Korth, H. F. and Sudarshan, S. (2020) Database System Concepts. 7th edn. New York: McGraw-Hill.
- Oracle Corporation (2024) MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/